

Dokumentace k IoT Teplotnímu Systému s Oboustrannou Komunikací

Multi-node architektura, filtrace telemetrie a adresné ovládání

Verze dokumentu: Produkční sestavení

Cílová skupina: Provozovatelé systému a vývojoví inženýři

Technologický zásobník: ESP32, SIM800L (GPRS), Node.js (TypeScript backend), SQLite, Chart.js, Vanilla JS

1. Představení systému

Tento dokument slouží jako komplexní technická a uživatelská dokumentace pro vestavěný (embedded) IoT systém monitorování teploty na více stanovištích současně. Architektura je navržena pro vysokou energetickou autonomii koncových měřicích uzlů a robustní distribuci dat přes mobilní celulární síť (GPRS) na centrální server.

Systém podporuje rozlišování stanovišť pomocí unikátních celočíselných identifikátorů (`device_id`) a textových názvů zařízení (`device_name`). Uživatelské webové rozhraní disponuje klientskou filtrací a umožňuje asynchronní řízení zpětné vazby (adresné blikání LED signalizace).

2. Uživatelská příručka (Webový Dashboard)

Webová stránka představuje centralizovaný dispečink, který vizualizuje naměřené stavy a umožňuje operátorovi analýzu historických i aktuálních trendů. Rozhraní je vyvedeno v tmavém režimu (Dark Mode) a veškerá interaktivní logika je napsána v čistém (vanilla) JavaScriptu.

2.1 Popis ovládacích prvků a navigace

Při otevření dashboardu se inicializuje spojení se serverem a stáhne se posledních 100 záznamů ze všech aktivních stanovišť. Hlavní obrazovka se skládá ze tří bloků:

1. **Panel nástrojů a filtrů:** Umožňuje omezit zobrazená data a vysílat příkazy hardwaru.
2. **Teplotní graf:** Vizualizace vývoje teploty v čase pro vybraná zařízení.
3. **Datová tabulka:** Textový výpis chronologicky seřazených měření.

2.2 Práce s teplotním grafem

Graf využívá knihovnu `Chart.js`. Pro zachování přesnosti prezentovaných dat je parametr interpolace křivky (tension) nastaven na hodnotu 0. Propojení jednotlivých bodů je tak přímé (lineární) bez zaoblení, což eliminuje optické zkreslení hodnot v mezidobí asynchronního odesílání dat.

- **Rozlišení zařízení:** Každé koncové zařízení má v grafu přiřazenu svou unikátní barvu.
- **Interaktivní legenda:** Kliknutím na název zařízení v horní legendě grafu lze danou křivku skrýt nebo zobrazit.
- **Kontinuita při spánku:** Graf je konfigurován tak, aby propojoval body linkou i v případě, že uzel odesílá data méně často (funkce `spanGaps: true`).

2.3 Používání filtrů

Filtrování probíhá na straně klienta po zadání hodnot a kombinuje parametry současně (logický operátor AND):

- **Filtr Zařízení:** Rozbalovací seznam (dropdown), ve kterém lze vybrat konkrétní měřicí uzel pro eliminaci překlepů.

- **Rozsah Teploty (°C):** Umožňuje zadat minimální a maximální mezní hodnoty. Tabulka skryje měření mimo tento rozsah.
- **Časový filtr (Datum a čas):** Omezuje výpis podle data a času. Pokud je zadán pouze čas, systém automaticky přednastaví vyhledávání na aktuální den.

2.4 Adresné ovládání LED (Zpětná vazba)

Systém umožňuje poslat příkaz k rozblikání vestavěné LED na fyzickém zařízení:

1. V levé části panelu nástrojů v rozbalovacím seznamu "Ovládat LED na zařízení:" vyberte požadovaný uzel (označený pomocí ID a názvu).
2. Klikněte na tlačítko  **Bliknout**.
3. Prohlížeč zobrazí potvrzení o zařazení příkazu do fronty. Při dalším probuzení a odeslání dat si zařízení příkaz převezme a provede blikací sekvenci.

3. Inženýrská dokumentace

3.1 Architektura systému

- **Koncový uzel:** Mikrokontrolér ESP32 se senzorem Dallas DS18B20 a modulem SIM800L. Komunikace probíhá přes UART pomocí AT příkazů, zapouzdřených do **externí HTTP implementace** (prostřednictvím `external_components`) v ESPHome.
- **Backend Server:** Aplikace běžící v prostředí Node.js s využitím frameworku Express. Pro zvýšení spolehlivosti kódu, udržitelnosti a bezpečnější definici REST API kontraktů je celý backend **vyvíjen v jazyce TypeScript** s přísnou kontrolou typů (strict mode) a typovými definicemi (např. `@types/express`). Před spuštěním serveru musí instalující uživatel kód ručně transpilovat pomocí kompilátoru (`tsc`) do nativního JavaScriptu (ECMAScript 2022). O datovou perzistenci se stará knihovna SQLite3.
- **Frontend:** Statická HTML5 stránka s klientským skriptováním postaveným na čistém (vanilla) JavaScriptu a externí knihovně Chart.js.

3.2 Databázové schéma

Využívá se SQLite databáze. Inicializace probíhá automaticky při startu serveru.

Tabulka: `mereni`

Sloupec	Datový typ	Atributy	Popis
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT	Interní index záznamu.
<code>device_id</code>	INTEGER	NOT NULL	Identifikátor fyzického zařízení.
<code>device_name</code>	TEXT	DEFAULT 'Nezname'	Název stanoviště.
<code>temperature</code>	REAL	NOT NULL	Naměřená hodnota teploty.
<code>timestamp</code>	DATETIME	DEFAULT CURRENT_TIMESTAMP	Čas uložení.

Tabulka: **prikazy**

Sloupec	Datový typ	Atributy	Popis
<code>device_id</code>	INTEGER	PRIMARY KEY	Identifikátor cílového zařízení.
<code>prikaz_blink</code>	INTEGER	DEFAULT 0	Požadavek na akci (1 = bliknutí, 0 = klid).

3.3 Specifikace REST API (Na základě TypeScript rozhraní)

Rozhraní jsou definována tak, aby reflektovala typová rozhraní TypeScriptu (např. `DataPostRequest`, `CommandBlinkRequest`).

1. **POST /data** (Odesílání telemetrie z ESP32)

Voláno koncovým zařízením po inicializaci GPRS spojení.

```
{"device_id": 2, "device_name": "SAGElab", "temperature": 23.4}
```

Odpověď (Status 201):

```
{"message": "Teplota uložena!", "blink": true}
```

2. **GET /data** (Načítání dat pro Dashboard)

Vrací JSON pole objektů seřazených sestupně podle času (limit 100 záznamů).

3. **POST /command/blink** (Zařazení příkazu z Webu)

```
{"device_id": 2}
```

4. **GET /data/filter** (Rozšířené backend filtrování dat)

Odpovídá rozhraní `FilterQuery`. Umožňuje pokročilé parametrizované filtrování nad SQLite a vrácení výsledků JSON (podporuje klíče ``device_id``, ``device_name``, ``temp_min``, ``temp_max``, ``date_from``, ``date_to``, ``limit``).

3.4 Firmware a konfigurace (ESPHome)

Měřicí uzly jsou konfigurovány pomocí YAML souborů pro ESPHome (soubor `esp.yaml`). Odesílání dat je řízeno blokem `on_boot`. Zpracování odpovědi využívá funkci `json::parse_json`. Aby byla šablona univerzální, jsou parametry specifické pro daný uzel nahrazeny zástupnými značkami (`<...>`).

```

esphome:
  name: esp32-<ESP32 Location>
  friendly_name: <ESP32 Location>
  min_version: 2026.4.0
  name_add_mac_suffix: false
  on_boot:
    priority: -100
    then:
      - delay: 65s
      - sim800l.http_post:
          url: "http://<IP_ADDRESS>/data"
          body: !lambda |-
              return sprintf("{\"device_id\": \"%d\", \"device_name\": \"%s\", \"temperature\": %.1f}", <DEVICE_ID>, <DEVICE_NAME>, id(temp).state);
          content_type: "application/json"

# ... (zbytek konfigurace viz repozitář)

```

Popis zástupných značek v YAMLu:

- **<ESP32 Location>**: Název uzlu bez mezer, který systém používá interně (např. `sklenik-01`).
- **<IP_ADDRESS>**: Veřejná IP adresa vašeho serveru včetně portu.
- **<DEVICE_ID>**: Unikátní celočíselný identifikátor uzlu pro databázi (např. `1`, `2`).
- **<DEVICE_NAME>**: Textový název uzlu. Protože je vkládán do kódu C++, **musí být obalen uvozovkami** (např. `"SAGElab"`).

4. Hardwarové zapojení komponent

Pro spolehlivou funkci uzlu je nutné zajistit odpovídající napájení a logické zapojení periférií. Následující konfigurace popisuje zapojení integrované vývojové desky (např. LILYGO TTGO T-Call), která v sobě kombinuje čip ESP32 a modem SIM800L.

4.1 Integrovaný modul ESP32 se SIM800L (LILYGO)

V případě použití vývojové desky s integrovaným modemem je fyzické propojení obou čipů z velké části vyřešeno již návrhem plošného spoje, což odpovídá definici v souboru `esp.yaml`:

- **Sériová linka (UART)**: Linky RX a TX modulu SIM800L jsou interně připojeny na piny **GPIO27 (TX)** a **GPIO26 (RX)** čipu ESP32. Není třeba je fyzicky propojovat vnějšími vodiči.
- **Napájení modulu**: Modem SIM800L je charakteristický vysokým odběrem (špičkově až 2A) během komunikace se sítí. Deska obsahuje vlastní měnič, lze ji proto pohodlně napájet standardním **kabelem přes USB-C konektor**.
- **Alternativní napájení**: Kromě USB-C portu je možné využít integrovaný bateriový konektor s přibaleným dvouvodičovým kabelem (JST) a modul napájet z externího bateriového článku (např. LiPo baterie).
- **Řízení napájení**: Zapínání napájecí větve pro samotný modem je softwarově řízeno sepnutím pinu **GPIO23** (id: `PWR`).

4.2 Teplotní senzor Dallas DS18B20 (Sběrnice OneWire)

DS18B20 je přesný digitální teploměr komunikující prostřednictvím 1-Wire sběrnice.

- **VDD** → **3.3V** (na desce ESP32)
- **GND** → **GND** (na desce ESP32)
- **DQ (Data)** → **GPIO33** (na desce ESP32)
- **Nezbytný Pull-up rezistor:** Pro spolehlivou logickou komunikaci na sběrnici 1-Wire je **nutné** vložit pull-up rezistor o hodnotě **4.7 kΩ** mezi datový vodič (DQ) a napájení (VDD).

4.3 Volitelná signalizační LED dioda

Pro vizualizaci zpětné vazby z webového rozhraní je **doporučeno** využít externí LED diodu. Upozorňujeme však, že zapojení této diody není pro základní fungování sběru teplotních dat povinné.

- **Anoda (delší vývod)** → **GPIO13** (na desce ESP32)
- **Katoda (kratší vývod)** → **GND** (na desce ESP32)
- **Ochranný rezistor:** Katodu (případně anodu) připojujte **striktně přes sériový omezovací rezistor**. Vzhledem k výstupnímu napětí 3.3V na pinu ESP32 je pro standardní signalizační LED diodu vyžadován předřadný rezistor s hodnotou **220 Ω až 330 Ω**. Přímé připojení bez rezistoru může vést k destrukci LED diody i k trvalému poškození výstupního pinu mikrokontroléru.